

LANGGRAPH MASTERY SERIES



PHASE 3

Agentic Patterns

Compose simple agents into complex architectures.
Supervisors, parallel fan-out, planning, and reflection.

WHAT YOU WILL MASTER

- Subgraphs: agents inside agents
- Multi-agent: the supervisor + worker pattern
- Map-reduce: parallel fan-out with the Send API
- Plan-and-execute: a planner + executor
- Reflection / self-critique feedback loops
- Agent handoffs and cross-graph communication
- LangGraph Studio for live visual debugging

Four projects: a supervisor, a parallel researcher, a self-correcting agent, a planner.

Built on the official LangGraph documentation (docs.langchain.com) · LangGraph v1.1, 2026

Composing agents into architectures

A single ReAct agent is powerful, but real systems are built from many cooperating parts: a coordinator that delegates, workers that specialise, parallel branches that fan out and back in, planners that lay out multi-step work, and critics that catch mistakes. This phase is a catalogue of the composition patterns that turn one agent into a robust architecture — and four projects that build the most important ones.

Two distinctions from the official documentation orient everything here. First, **workflows** have predetermined paths while **agents** decide their own — most architectures mix both. Second, when people say “multi-agent,” they usually want one of three things: **context management** (give each part only what it needs), **distributed development** (teams own components independently), or **parallelization** (run sub-tasks concurrently). Pick your pattern to match the need.

1. Subgraphs: agents inside agents

A **subgraph** is a compiled graph used as a node inside a larger graph. This is the foundation of composition: build and test a small agent in isolation, then drop it into a bigger system as a single node. The parent need not know the subgraph's internal structure — only its state interface.

A compiled graph plugged into a parent as a single node.

```
from langgraph.graph import StateGraph, START, MessagesState

# Build and compile a small agent (the subgraph) on its own:
sub = StateGraph(MessagesState)
# ... add nodes/edges ...
subgraph = sub.compile()

# Use it as ONE node in a parent graph:
parent = StateGraph(MessagesState)
parent.add_node("researcher", subgraph)      # a compiled graph IS a node
parent.add_edge(START, "researcher")
graph = parent.compile()
```

- **Shared state keys flow automatically.** If parent and subgraph share a state key, values pass through transparently.
- **Checkpoint namespaces** keep subgraph state separate: the parent uses namespace "", a subgraph uses "node_name:uuid".
- **Stream from subgraphs** by passing subgraphs=True; each event carries an ns tuple identifying its source graph.

When to reach for a subgraph

Use a subgraph whenever a piece of your system is independently useful, independently testable, or owned by a different team. It is the unit of reuse and the boundary of context — exactly what multi-agent systems are made of.

2. Multi-agent: the supervisor + worker pattern

In the **supervisor** pattern, a coordinator agent routes work to specialised **worker** agents and synthesises their results. The crucial design rule, learned the hard way in production: **the supervisor routes, it does not do the work**. It is a traffic controller. The LLM decides *what* to do; deterministic Python code decides *how and when to stop*.

The rule that prevents infinite loops

Never enforce iteration limits in the prompt ("stop after 5 turns" will be ignored). Enforce them in code: if `state['iteration'] > N`: route to END. The LLM chooses the next worker; your code guarantees termination.

LangGraph supports several multi-agent patterns. The table summarises when each fits, drawn from the official comparison:

Pattern	How it works	Strong at
Supervisor / Subagents	A coordinator invokes workers (as tools or subgraphs); results flow back through it.	Parallelization, context isolation, multi-hop
Handoffs	Agents transfer control to each other via tool calls that update routing state.	Direct user interaction, stateful conversations
Router	A classifier step directs input to one specialised agent; results synthesised.	Clean single-domain routing, parallel fan-out
Custom workflow	Hand-built graph mixing deterministic and agentic nodes; embed other patterns as nodes.	Full control over the execution flow

A minimal supervisor wires worker subgraphs as nodes and uses a routing function (or a Command) to choose the next worker based on the LLM's decision:

A supervisor that delegates to worker subgraphs and re-collects control after each.

```
from typing import Literal
from langgraph.graph import StateGraph, MessagesState, START, END
from langgraph.types import Command

def supervisor(state: MessagesState) -> Command[Literal["researcher", "writer", "__end__"]]:
    # The LLM decides which worker is needed next (or that we're done).
    decision = route_llm.invoke(state["messages"]) # returns 'researcher'|'writer'|'done'
    if decision == "done":
        return Command(goto=END)
    return Command(goto=decision)

builder = StateGraph(MessagesState)
builder.add_node("supervisor", supervisor)
builder.add_node("researcher", researcher_subgraph) # workers are subgraphs
builder.add_node("writer", writer_subgraph)
builder.add_edge(START, "supervisor")
builder.add_edge("researcher", "supervisor") # workers report back
builder.add_edge("writer", "supervisor")
graph = builder.compile()
```

3. Map-reduce: parallel fan-out with the Send API

When you must run the *same* operation over a list whose length is unknown until runtime — research five sub-topics, grade a document on three criteria, summarise N files — you want **map-reduce**: fan out to many parallel workers (map), then combine their outputs (reduce).

LangGraph's Send API does the fan-out dynamically.

A conditional edge returns a list of `Send(node, payload)` objects. LangGraph spawns one instance of node per Send, each with its own payload, all in parallel. Their outputs accumulate into a shared list via an operator `.add reducer` (the reduce step).

Map-reduce via Send: dynamic parallel fan-out, then fan-in through an add reducer.

```
import operator
from typing import Annotated
from typing_extensions import TypedDict
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send

class State(TypedDict):
    topic: str
    subjects: list[str]
    jokes: Annotated[list[str], operator.add]  # reducer: workers append here

def generate_subjects(state: State):
    return {"subjects": ["lions", "elephants", "penguins"]}

def generate_joke(state: dict):  # one WORKER instance per subject
    return {"jokes": [f"A joke about {state['subject']}"]}

# MAP: fan out – one Send per subject, run in parallel
def assign_workers(state: State):
    return [Send("generate_joke", {"subject": s}) for s in state["subjects"]]

def best_joke(state: State):  # REDUCE: combine accumulated jokes
    return {"best": pick_best(state["jokes"])}

b = StateGraph(State)
b.add_node("generate_subjects", generate_subjects)
b.add_node("generate_joke", generate_joke)
b.add_node("best_joke", best_joke)
b.add_edge(START, "generate_subjects")
b.add_conditional_edges("generate_subjects", assign_workers, ["generate_joke"])
b.add_edge("generate_joke", "best_joke")
b.add_edge("best_joke", END)
graph = b.compile()
```

Send vs. static parallel edges

If the number of branches is FIXED and known at build time, use plain parallel edges (`START→A`, `START→B`, `START→C`, all `→ aggregator`). Use Send when the number of branches depends on runtime state — that is the case a static graph cannot express.

Ordering and exceptions

Parallel updates may not arrive in a deterministic order — if you need ordering, include a sort key in each worker's output. A super-step is transactional: if one parallel branch raises, none of that step's updates apply (unless a retry policy recovers the failing branch).

4. Plan-and-execute: a planner node + executor

For multi-step tasks, a single ReAct loop can lose the thread. The **plan-and-execute** pattern (LangGraph calls the general form **orchestrator-worker**) separates thinking from doing: a **planner** breaks the task into sub-tasks, then **workers** execute each, and a **synthesizer** assembles the result. Unlike fixed parallelization, the plan is generated dynamically, so the system adapts to each task.

Plan-and-execute: a planner emits sections, workers write them in parallel, a synthesizer joins them.

```
from typing import Annotated, List
import operator
from pydantic import BaseModel, Field
from langchain.messages import SystemMessage, HumanMessage
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send

class Section(BaseModel):
    name: str = Field(description="Section title.")
    description: str = Field(description="What this section should cover.")

class Plan(BaseModel):
    sections: List[Section]

planner = llm.with_structured_output(Plan)  # structured planning output

class State(TypedDict):
    topic: str
    sections: list
    completed: Annotated[list, operator.add]  # workers append finished sections
    final_report: str

def orchestrator(state: State):  # PLAN
    plan = planner.invoke([
        SystemMessage("Generate a plan for the report."),
        HumanMessage(f"Topic: {state['topic']}"),
    ])
    return {"sections": plan.sections}

def assign(state: State):  # fan out one worker per section
    return [Send("write_section", {"section": s}) for s in state["sections"]]

def write_section(state: dict):  # EXECUTE
    section = state["section"]
    result = llm.invoke([
        SystemMessage("Write this report section in markdown."),
        HumanMessage(f"{section.name}: {section.description}"),
    ])
    return {"completed": [result.content]}

def synthesizer(state: State):  # COMBINE
    return {"final_report": "\n\n---\n\n".join(state["completed"])}

b = StateGraph(State)
for name, fn in [("orchestrator", orchestrator),
                ("write_section", write_section),
                ("synthesizer", synthesizer)]:
    b.add_node(name, fn)
b.add_edge(START, "orchestrator")
b.add_conditional_edges("orchestrator", assign, ["write_section"])
b.add_edge("write_section", "synthesizer")
b.add_edge("synthesizer", END)
report_graph = b.compile()
```

This is also a clean example of mixing patterns: the planner is agentic (it decides the sections), while the fan-out/fan-in is a deterministic map-reduce workflow. That blend — agentic where you need adaptability, deterministic where you need control — is the hallmark of good architecture.

5. Reflection / self-critique loops

Models produce better output when they critique and revise it. The **reflection** pattern (the docs call it **evaluator-optimizer**) wires a generator and an evaluator into a loop: the generator produces a draft, the evaluator grades it and gives feedback, and if it falls short the draft is regenerated *with that feedback* — repeating until the bar is met.

A reflection loop: generate -> evaluate -> (revise or finish). The feedback is fed back into generation.

```
from typing import Literal
from pydantic import BaseModel, Field
from langgraph.graph import StateGraph, START, END

class Feedback(BaseModel):
    grade: Literal["pass", "fail"] = Field(description="Does the draft meet the bar?")
    feedback: str = Field(description="If failing, how to improve it.")

evaluator = llm.with_structured_output(Feedback)

class State(TypedDict):
    topic: str
    draft: str
    feedback: str
    grade: str

def generate(state: State):
    # OPTIMIZER
    prompt = f"Write about {state['topic']}."
    if state.get("feedback"):
        prompt += f" Incorporate this feedback: {state['feedback']}"
    return {"draft": llm.invoke(prompt).content}

def evaluate(state: State):
    # EVALUATOR
    verdict = evaluator.invoke(f"Grade this draft:\n{state['draft']}")
    return {"grade": verdict.grade, "feedback": verdict.feedback}

def route(state: State) -> Literal["generate", "__end__"]:
    return END if state["grade"] == "pass" else "generate" # loop until pass

b = StateGraph(State)
b.add_node("generate", generate)
b.add_node("evaluate", evaluate)
b.add_edge(START, "generate")
b.add_edge("generate", "evaluate")
b.add_conditional_edges("evaluate", route, {"generate": "generate", "__end__": END})
graph = b.compile()
```

Always bound the loop

A reflection loop can oscillate. Add a hard cap — track an attempt counter in state and force END after N rounds, and set a recursion_limit as a backstop. A perfectionist evaluator with no cap is an infinite-cost machine.

Human-in-the-loop reflection

Replace (or augment) the LLM evaluator with a human via `interrupt()` from Phase 2. The human reads the draft and supplies the feedback that drives the next revision — a powerful pattern for high-stakes content.

6. Agent handoffs and cross-graph communication

In the **handoff** pattern, control passes directly from one agent to another rather than always routing back through a coordinator. An agent decides, via a tool call, to hand the conversation to a specialist. Handoffs shine for direct user interaction and stateful, multi-turn conversations because the active agent persists between turns — no re-routing overhead on repeat requests.

Mechanically, a handoff is a `Command` that both updates state and routes to another agent. When a worker (a subgraph node) needs to jump to a node in the *parent* graph, it uses `Command(graph=Command.PARENT)` — the primitive for cross-graph communication.

Handoffs and cross-graph routing with `Command`. `PARENT` lets a subgraph node target the parent.

```
from langgraph.types import Command
from typing import Literal

def specialist_router(state) -> Command[Literal["billing_agent", "tech_agent"]]:
    target = decide_specialist(state)      # e.g. based on the user's issue
    return Command(update={"handoff_reason": "..."}, goto=target)

# From inside a SUBGRAPH node, route to a node in the PARENT graph:
def worker_node(state) -> Command:
    return Command(
        update={"result": "..."},
        goto="next_parent_node",
        graph=Command.PARENT,             # hop up to the parent graph
    )
```

Shared keys need reducers

When a subgraph node updates a state key that the parent also owns (via `Command.PARENT`), define a reducer for that key in the parent state — otherwise concurrent writes from the hop can conflict.

7. LangGraph Studio: see and debug your graph live

As architectures grow, reading code is no longer enough — you need to see the graph run. **LangGraph Studio** (part of LangSmith) is a visual IDE for agents. It renders your graph, lets you run it interactively, inspect the state at every node, set breakpoints, and resume — turning the abstract execution into something you can watch and poke.

You run it against a local development server. The workflow:

- 1 Add a `langgraph.json` manifest that points at your compiled graph(s).
- 2 Start the dev server with the CLI: `langgraph dev` (installs with `pip install -U "langgraph-cli[inmem]"`).
- 3 Open Studio in the browser. Submit inputs, watch each node light up, and inspect the evolving state.

4 Set **static interrupts** (breakpoints) before or after any node directly in the UI to step through execution one super-step at a time.

A minimal `langgraph.json`. ``langgraph dev`` reads this to serve your graph to Studio.

```
// langgraph.json – tells the dev server where your graph lives
{
  "dependencies": ["."],
  "graphs": {
    "agent": "./my_agent.py:graph"    // module path : variable name
  },
  "env": ".env"
}
```

Studio + interrupts = the best debugger

Combine Studio's visual breakpoints with the `get_state_history()` time-travel from Phase 2: pause at a node, inspect state, rewind to an earlier checkpoint, edit it, and replay a different path. This is the fastest way to understand why an agent did something surprising.

Project 1 — A supervisor with a researcher and a writer

PROJECT 1

Coordinate two specialist workers from a supervisor

Goal: Build a supervisor that delegates to a 'researcher' worker (gathers facts) and a 'writer' worker (turns facts into prose), then decides when the task is done. The supervisor routes; the workers work.

```
from typing import Literal
from typing_extensions import TypedDict
from langchain.chat_models import init_chat_model
from langchain.messages import SystemMessage, HumanMessage
from langgraph.graph import StateGraph, START, END
from langgraph.types import Command

llm = init_chat_model("claude-sonnet-4-6")

class State(TypedDict):
    task: str
    research: str
    draft: str
    next: str

# --- Workers (kept simple; each could be a full subgraph) ---
def researcher(state: State):
    notes = llm.invoke([
        SystemMessage("You are a researcher. Produce concise factual bullet points."),
        HumanMessage(state["task"]),
    ]).content
    return {"research": notes}

def writer(state: State):
    draft = llm.invoke([
        SystemMessage("You are a writer. Use the research to write a tight 1-paragraph answer."),
        HumanMessage(f"Task: {state['task']}\n\nResearch:\n{state['research']}"),
    ]).content
    return {"draft": draft}

# --- Supervisor: routes, does not do the work ---
def supervisor(state: State) -> Command[Literal["researcher", "writer", "__end__"]]:
    if not state.get("research"):
        return Command(goto="researcher")    # no research yet -> research
    if not state.get("draft"):
        return Command(goto="writer")        # have research, no draft -> write
    return Command(goto=END)                # both done -> finish

b = StateGraph(State)
b.add_node("supervisor", supervisor)
b.add_node("researcher", researcher)
b.add_node("writer", writer)
b.add_edge(START, "supervisor")
b.add_edge("researcher", "supervisor")    # report back to the supervisor
b.add_edge("writer", "supervisor")
team = b.compile()
```

```
out = team.invoke({"task": "Explain why LangGraph uses graphs instead of chains.",
                  "research": "", "draft": "", "next": ""})
print(out["draft"])
```

The routing is deterministic Python here for clarity; in a larger system the supervisor would use an LLM to pick the next worker. The key lesson holds either way: **the supervisor never writes or researches itself** — it only decides who goes next and when to stop.

Project 2 — A parallel web-research agent (map-reduce)

PROJECT 2

Fan out research across sub-questions, then synthesise

Goal: Given a question, generate several sub-questions, research each in parallel with Send, and combine the findings into one answer. This is the map-reduce pattern applied to research.

```
import operator
from typing import Annotated, List
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langchain.messages import SystemMessage, HumanMessage
from langgraph.graph import StateGraph, START, END
from langgraph.types import Send

llm = init_chat_model("claude-sonnet-4-6")

class SubQs(BaseModel):
    questions: List[str] = Field(description="3-5 focused sub-questions.")

planner = llm.with_structured_output(SubQs)

class State(TypedDict):
    question: str
    sub_questions: list
    findings: Annotated[list, operator.add] # parallel workers append here
    answer: str

def plan(state: State): # decompose
    qs = planner.invoke([
        SystemMessage("Break the question into focused, independent sub-questions."),
        HumanMessage(state["question"]),
    ])
    return {"sub_questions": qs.questions}

def fan_out(state: State): # MAP: one researcher per sub-question
    return [Send("research", {"q": q}) for q in state["sub_questions"]]

def research(state: dict): # worker (swap in a real search tool)
    note = llm.invoke(f"Answer concisely with key facts: {state['q']}").content
    return {"findings": [f"Q: {state['q']}\nA: {note}"]}

def synthesize(state: State): # REDUCE
    joined = "\n\n".join(state["findings"])
    final = llm.invoke([
        SystemMessage("Synthesize these findings into one coherent answer."),
        HumanMessage(f"Question: {state['question']}\n\nFindings:\n{joined}"),
    ]).content
    return {"answer": final}

b = StateGraph(State)
b.add_node("plan", plan)
b.add_node("research", research)
```

```
b.add_node("synthesize", synthesize)
b.add_edge(START, "plan")
b.add_conditional_edges("plan", fan_out, ["research"])
b.add_edge("research", "synthesize")
b.add_edge("synthesize", END)
researcher = b.compile()

out = researcher.invoke({"question": "What makes LangGraph suitable for production agents?",
                        "sub_questions": [], "findings": [], "answer": ""})
print(out["answer"])
```

Because the sub-questions are researched in parallel, total latency is roughly the slowest single branch rather than the sum of all branches. Swap the placeholder research body for a real web-search tool and you have a genuine parallel research agent.

Project 3 — A self-correcting agent (reflection loop)

PROJECT 3

Generate, critique, and revise until it passes

Goal: Build a generator-evaluator loop that drafts an answer, grades it against criteria, and revises using the feedback — bounded by a max-attempts guard so it always terminates.

```
from typing import Literal
from typing_extensions import TypedDict
from pydantic import BaseModel, Field
from langchain.chat_models import init_chat_model
from langgraph.graph import StateGraph, START, END

llm = init_chat_model("claude-sonnet-4-6")

class Verdict(BaseModel):
    grade: Literal["pass", "fail"] = Field(description="Meets the bar?")
    feedback: str = Field(description="Specific, actionable improvements if failing.")

evaluator = llm.with_structured_output(Verdict)

class State(TypedDict):
    task: str
    draft: str
    feedback: str
    grade: str
    attempts: int

def generate(state: State):
    prompt = f"Task: {state['task']}\nWrite the best possible answer."
    if state.get("feedback"):
        prompt += f"\nRevise using this feedback: {state['feedback']}"
    return {"draft": llm.invoke(prompt).content,
            "attempts": state.get("attempts", 0) + 1}

def evaluate(state: State):
    v = evaluator.invoke(
        f"Grade this answer for correctness and clarity:\n{state['draft']}"
    )
    return {"grade": v.grade, "feedback": v.feedback}

def route(state: State) -> Literal["generate", "__end__"]:
    # Pass OR ran out of attempts -> stop. Otherwise revise.
    if state["grade"] == "pass" or state["attempts"] >= 3:  # hard cap
        return END
    return "generate"

b = StateGraph(State)
b.add_node("generate", generate)
b.add_node("evaluate", evaluate)
b.add_edge(START, "generate")
b.add_edge("generate", "evaluate")
b.add_conditional_edges("evaluate", route, {"generate": "generate", "__end__": END})
critic = b.compile()
```

```
out = critic.invoke({"task": "Explain checkpoints in LangGraph in 3 sentences.",
                    "draft": "", "feedback": "", "grade": "", "attempts": 0})
print(f"Passed after {out['attempts']} attempt(s):\n{out['draft']}")
```

The `attempts >= 3` guard is non-negotiable. Without it, a strict evaluator could reject every draft forever. With it, the agent self-improves up to a fixed budget and then ships the best version it has.

Project 4 — A plan-and-execute agent for multi-step tasks

PROJECT 4

Plan dynamically, execute in parallel, synthesise

Goal: Combine everything: a planner decomposes a report request into sections, workers write each section in parallel via Send, and a synthesizer assembles the final document. Then view it live in LangGraph Studio.

Running the planner end to end, then handing it to Studio to watch the parallel fan-out live.

```
# Reuse the plan-and-execute graph from section 4 (orchestrator -> Send -> synthesizer).
# Here we run it and then expose it to LangGraph Studio for visual debugging.

report = report_graph.invoke(
    {"topic": "A beginner's guide to LangGraph state and reducers",
     "sections": [], "completed": [], "final_report": ""}
)
print(report["final_report"])

# To debug visually:
# 1) Save the graph as `report_graph` in report_agent.py
# 2) Create langgraph.json:
#     { "dependencies": ["."], "graphs": { "report": "./report_agent.py:report_graph" } }
# 3) Run: langgraph dev
# 4) Open Studio, submit a topic, and watch orchestrator -> N parallel writers -> synthesizer.
```

In Studio you will see the orchestrator node fire once, then a burst of parallel `write_section` instances (one per planned section), then the synthesizer collecting them. Set a breakpoint after the orchestrator to inspect the generated plan before any section is written — a perfect demonstration of why visual debugging matters for branching architectures.

Phase 3 checkpoint

You can now compose agents with subgraphs, coordinate them with a supervisor, fan out and back in with Send (map-reduce), separate planning from execution, close the loop with reflection, route with handoffs and Command.PARENT, and debug it all in Studio. You have built four real architectures. The final step is making them production-grade.

Coming in Phase 4 — Production-Grade Agents: LangSmith tracing for full observability, evaluation with datasets and LLM-as-judge, deployment via LangGraph Platform as a REST API, long-running and background tasks, fault tolerance with retries and fallbacks, auth and per-user tool scoping, versioning, and cost/latency optimisation — culminating in a capstone end-to-end agentic system.